

Code review in the agentic era: an intern learning path

Overall goal: understand what code review is, why it became a bottleneck once AI agents started writing the code, and what ideas exist for speeding it up. You're not building anything production-grade here. Your job is to think about the concept and form your own opinion.

Rough time: about 4 to 5 working days. Every task has a concrete deliverable. You don't need to know any of these terms at the start. That's what the tasks are for.

Task 0: warm-up, what even is code review?

Goal: understand the basics before we touch anything about AI.

What to do:

Imagine you wrote a piece of text, and before publishing it a colleague reads it and says "typo here, this sentence is unclear, and you got this fact wrong." Code review is exactly that, but for code.

Find out (Google or the GitHub docs) what a pull request (PR) and a diff are. These two terms are the foundation for everything else.

Create a free GitHub account. Find any open-source project, go to the "Pull requests" tab, open one, and just look. Notice the green and red lines (the diff), the reviewers' comments, the discussion.

Deliverable: in your own words (half a page max), answer these. What is code review and why do people do it? What is a pull request and a diff? What benefits does review give, and what costs or annoyances does it have? Guess on that last one, we'll check it later.

Task 1: why review exists

Goal: understand that review isn't bureaucracy. It's a tool for catching problems.

What to do:

List the kinds of problems a reviewer can catch: logic bugs, security holes, unreadable code, missing tests, breaking the team's conventions, and so on.

Then ask yourself which of these a machine can check mechanically (formatting, say) and which ones need an understanding of the context of the whole project.

Deliverable: a two-column table. One column for "what can be checked automatically with rules," the other for "what requires understanding the broader context." This distinction comes back later when we look at tools.

Task 2: where the "agentic era" problem came from

Goal: understand why this became a hot topic right now.

What to do:

Find out what an AI coding agent is (Claude Code, Cursor, Codex, OpenCode are examples). It's a tool that writes and changes code itself, not just suggests.

Then think through a simple cause-and-effect chain and write it down. An agent can generate 1,000 lines of code in 5 minutes. A human reviewing one line takes about as long as it did a year ago. So what happens to the queue of PRs waiting to be reviewed?

That's the crux. The speed of generating code has exploded, while the speed of human review hasn't. A bottleneck forms.

Deliverable: a paragraph (5 to 8 sentences) explaining to a friend outside the industry why, even though AI writes code faster, review suddenly became the problem. If you can explain it simply, you understand the concept.

Task 3: survey of existing ideas for speeding it up

Goal: see what the world has already built, and group the approaches.

What to do: look up the following and assign each one to a category. You decide which it fits.

Category A is an agent that reviews the PR on the human's behalf. Examples include CodeRabbit, Greptile, Qodo, and GitHub Copilot Code Review. Open-source options are PR-Agent and Kodus.

Category B is changing the way of working, or a different UI. Stacked PRs (Graphite) is a good example: you split one giant PR into a stack of small, easier-to-evaluate ones. Cloud agents that open PRs themselves also fit here.

Category C is the deterministic layer, meaning rules rather than AI. Think Semgrep for security, SonarQube for quality, and linters.

One thing worth noticing: these don't exclude each other, they layer. That's an important intuition. There's no single best tool, there's a stack.

Deliverable: a short note with one sentence per tool (what it does) plus which category you put it in and why.

Task 4: touch it with your hands

Goal: feel how this actually works, not just read about it.

What to do, picking at least one of these:

The easier route is to connect a free AI reviewer (CodeRabbit's free tier, for instance) to a test repo on GitHub. Create a PR with an intentional bug, like a division by zero or a typo in a variable name, and see whether and how the tool catches it.

The harder route is to run the open-source PR-Agent or Kodus locally with your own model key. See how much effort the configuration takes. That's a valuable observation in itself.

Deliverable: 3 to 5 screenshots of the tool's comments, plus a note on what it did well, what it missed, and what was just noise (unnecessary comments). Pay attention to that noise. It's one of the main problems with these tools.

Task 5: think critically (the most important task)

Goal: form your own opinion. There's no single right answer here.

Questions to think through and write up:

If AI reviews code that another AI wrote, who takes responsibility for a bug in production?
Where is the human in the loop?

Is it better to get 50 comments from a bot (surely something useful is in there) or 5 of them (but the important ones)? Justify your answer.

Are there things AI review shouldn't do on its own, without a human? Which ones?

Go back to your table from Task 1. Does AI move the line between "automatically checkable" and "requires context"? In which direction?

Deliverable: one page of reflections. Honesty beats correctness. If you don't know something, write "I don't know, but I suspect that..."

Task 6 (stretch, optional): mini-presentation

Goal: check whether you can pass this on to others.

What to do: prepare 5 to 7 slides, or a 10-minute talk for the team. Walk through what code review is, why it became a bottleneck, what categories of solutions exist, what you tested yourself, and your recommendation or takeaway.

Deliverable: slides or talk notes.

How I'll evaluate this

So you know what to focus on:

I'm not looking at whether you know the "correct" tool names. I'm looking at whether you understand the problem (the bottleneck) and why the various ideas address it.

I value Tasks 2 and 5 the most. That's whether you can explain the concept simply and think about it critically.

If you get stuck on any term, message me right away instead of losing half a day. "What does X mean" is a welcome question here, not a stupid one.